
UNIVERSITY OF VERSAILLES SAINT-QUENTIN-EN-YVELINES
PARIS-SACLAY UNIVERSITY
Master's Degree in DataScale — Large-Scale Data Management

M2 Internship Notes : Graph Neural Networks (GCN, GAT, and GIN)

PREPARED BY
FIDÈLE OUEDRAOGO

SUPERVISORS
Prof. CHRISTINE LARGERON
Dr. HAIFEI ZHANG

DATE
7 March 2026

Hubert Curien Laboratory
2025–2026

Contents

1	Background	3
1.1	Graph Representation Learning	3
1.1.1	Traditional Methods	3
1.1.2	Graph Neural Networks	5
2	Graph Neural Network Models	7
2.1	From Basic GNN to Advanced Architectures	7
2.1.1	Basic Graph Neural Network	7
2.1.2	Graph Convolutional Network (GCN)	8
2.1.3	Graph Attention Network (GAT)	8
2.1.4	Graph Isomorphism Network (GIN)	9
3	Experimental Study	11
3.1	Objective	11
3.2	Experimental Environment	11
3.2.1	Hardware and Software Environment	11
3.3	Datasets	12
3.3.1	Datasets from Original Papers	12
3.3.2	Other Datasets	13
3.4	Evaluation Metric	13
3.4.1	Accuracy	14
4	Experimental Results	15
4.1	Reproduction of GCN Results	15
4.2	Reproduction of GAT Results	15
4.3	Reproduction of GIN Results	16

Introduction

Relational data are ubiquitous in numerous domains, including social networks, citation networks, recommender systems, biological networks, and transportation systems. Unlike traditional tabular data, they are naturally represented as graphs, where nodes model entities and edges represent the relationships between them. This relational structure makes graph analysis particularly challenging, as meaningful information arises not only from the attributes of individual nodes but also from the topology of the graph itself.

Learning from graph-structured data has become a fundamental problem in machine learning, with applications to several tasks such as node classification, link prediction, graph classification, and community detection. For many years, these tasks relied primarily on manually engineered graph features or shallow graph embedding techniques. Although these approaches achieved promising results in certain applications, they suffer from several limitations, particularly their inability to effectively leverage both node attributes and the structural information of the graph in a unified learning framework.

Graph Neural Networks (GNNs) were introduced to overcome these limitations by learning node and graph representations directly from both feature information and graph topology through neighborhood aggregation. Today, GNNs have become one of the most successful frameworks for graph representation learning and have inspired numerous architectures. Among them, the Graph Convolutional Network (GCN), the Graph Attention Network (GAT), and the Graph Isomorphism Network (GIN) are three of the most influential models and constitute the main focus of this study [1–3].

Chapter 1

Background

1.1 Graph Representation Learning

Graph representation learning aims at designing methods that can transform graph-structured data into meaningful vector representations while preserving both structural and attribute information. A graph is formally defined as $G = (V, E)$, where V is the set of nodes (vertices) and E is the set of edges representing relationships between nodes. Graphs provide a natural way to model relational systems such as social networks, citation networks, biological networks, or recommender systems.

Unlike traditional tabular data, graph data introduces additional complexity due to the dependencies between nodes. This makes learning tasks more challenging, as information is not only contained in individual node features but also in the connectivity pattern of the graph.

Graph learning methods are generally evaluated on several standard tasks, including node classification, link prediction, graph classification, and community detection. These tasks require models capable of capturing both local and global structural patterns.

1.1.1 Traditional Methods

Before the emergence of deep learning approaches on graphs, graph analysis relied heavily on manually designed features and shallow embedding techniques.

Graph Statistics

Early approaches focused on extracting structural statistics from graphs. Common measures include node degree, centrality, and clustering coefficient.

The degree of a node u is defined as:

$$d_u = \sum_v A[u, v]$$

where A is the adjacency matrix of the graph.

Centrality measures, such as eigenvector centrality, capture the importance of a node based on the importance of its neighbors:

$$e_u = \frac{1}{\lambda} \sum_v A[u, v] e_v$$

The clustering coefficient measures how connected the neighbors of a node are:

$$c_u = \frac{\text{number of triangles around } u}{\text{number of possible triangles}}$$

These features are simple but limited, as they are manually engineered and fixed for all tasks.

Graph Laplacian

Another fundamental tool in graph theory is the graph Laplacian, defined as:

$$L = D - A$$

where D is the degree matrix and A is the adjacency matrix.

The Laplacian matrix encodes the structure of the graph and is widely used in spectral clustering. Its eigenvalues and eigenvectors provide important insights into graph connectivity. In particular, the smallest eigenvalues correspond to connected components, while the second smallest eigenvector is used for graph partitioning (min-cut relaxation problem).

Node Embedding Methods

To overcome the limitations of handcrafted features, node embedding methods were introduced to learn continuous vector representations of nodes. The goal is to map each node u to a vector $z_u \in \mathbb{R}^d$ such that structurally similar nodes are close in the embedding space.

Formally, embeddings are learned through an encoder-decoder framework:

Encoder:

$$enc(u) = z_u$$

Decoder:

$$dec(z_u, z_v) \rightarrow similarity(u, v)$$

The objective is to reconstruct a similarity matrix S :

$$L = \sum (dec(z_u, z_v) - S[u, v])^2$$

Methods such as DeepWalk and node2vec define similarity based on random walks:

$$P(v|u)$$

and aim to learn embeddings such that:

$$z_u \cdot z_v \approx P(v|u)$$

These methods are equivalent to implicit matrix factorization of a graph-based co-occurrence matrix.

However, these approaches suffer from important limitations: they do not use node features, do not share parameters across nodes, and are not inductive.

Limitations of Traditional Methods

Despite their effectiveness in certain scenarios, traditional graph learning methods have several limitations:

- handcrafted features lack adaptability,
- shallow embeddings ignore node attributes,
- models are transductive and do not generalize to unseen nodes,
- structural and feature information are not jointly learned.

These limitations motivated the development of Graph Neural Networks.

1.1.2 Graph Neural Networks

Graph Neural Networks (GNNs) were introduced to overcome the limitations of traditional methods by learning representations directly from graph structure and node features through neural architectures.

General Principle

The key idea behind GNNs is to compute node representations by iteratively aggregating information from neighboring nodes. Unlike shallow embeddings, where each node has an independent parameter vector, GNN embeddings are computed dynamically from the graph structure.

This allows the model to capture both local and global dependencies in the graph.

Message Passing Framework

Most GNN architectures follow the message passing paradigm. At each layer k , each node u performs two operations:

Aggregation:

$$m_{N(u)}^{(k)} = \text{AGGREGATE}^{(k)}(\{h_v^{(k)} \mid v \in N(u)\})$$

Update:

$$h_u^{(k+1)} = \text{UPDATE}^{(k)}(h_u^{(k)}, m_{N(u)}^{(k)})$$

The initial node representations are given by:

$$h_u^{(0)} = x_u$$

After K layers, the final embedding is:

$$z_u = h_u^{(K)}$$

Each layer extends the receptive field of a node: - 1 layer $\rightarrow$ 1-hop neighbors - 2 layers $\rightarrow$ 2-hop neighbors - K layers $\rightarrow$ K -hop neighborhood

Advantages over Traditional Methods

Compared to classical embedding methods, GNNs offer several advantages:

- they jointly learn structure and features,
- they share parameters across nodes,
- they support inductive learning,
- they naturally generalize to unseen graphs or nodes.

Overall, GNNs provide a unified framework for learning on graph-structured data through iterative information propagation between nodes.

Chapter 2

Graph Neural Network Models

2.1 From Basic GNN to Advanced Architectures

2.1.1 Basic Graph Neural Network

Before introducing specialized architectures such as GCN or GAT, it is important to understand the fundamental idea behind Graph Neural Networks (GNNs). A basic GNN defines node representations by iteratively aggregating information from neighboring nodes.

At layer k , the representation of a node u is updated as follows:

$$h_u^{(k)} = \sigma \left(W_{\text{self}}^{(k)} h_u^{(k-1)} + W_{\text{neigh}}^{(k)} \sum_{v \in N(u)} h_v^{(k-1)} + b^{(k)} \right)$$

his formulation shows that each node combines: its own previous representation and the aggregated information from its neighbors.

In matrix form, this can be written as:

$$H^{(k)} = \sigma \left(A H^{(k-1)} W_{\text{neigh}}^{(k)} + H^{(k-1)} W_{\text{self}}^{(k)} \right)$$

A common variant introduces self-loops, allowing each node to explicitly include its own information in the aggregation process:

$$H^{(k)} = \sigma \left((A + I) H^{(k-1)} W^{(k)} \right)$$

where I is the identity matrix.

To ensure numerical stability and avoid exploding representations, a normalization step is often applied:

$$H^{(k)} = \sigma \left(\tilde{D}^{-1/2} \tilde{A} \tilde{D}^{-1/2} H^{(k-1)} W^{(k)} \right)$$

where $\tilde{A} = A + I$ and $\tilde{D}$ is its degree matrix.

This normalization prevents nodes with high degrees from dominating the aggregation process and improves training stability.

Key idea: A basic GNN updates node representations by combining self-information and neighborhood information through learned transformations and aggregation.

2.1.2 Graph Convolutional Network (GCN)

The Graph Convolutional Network (GCN) is a simplified and well-optimized version of the basic GNN framework. It introduces a symmetric normalization of the adjacency matrix, leading to a more stable and efficient learning process.

The propagation rule of a GCN layer is defined as:

$$H^{(k)} = \sigma \left(\tilde{D}^{-1/2} \tilde{A} \tilde{D}^{-1/2} H^{(k-1)} W^{(k)} \right)$$

where:

- $\tilde{A} = A + I$ (graph with self-loops),
- $\tilde{D}$ is the degree matrix of $\tilde{A}$,
- $W^{(k)}$ is a learnable weight matrix,
- σ is a nonlinear activation function.

Intuition: Each node aggregates a normalized average of its neighbors and itself, followed by a linear transformation and a non-linear activation.

GCN can be interpreted as:

- message passing,
- self-loop aggregation,
- degree normalization,
- feature transformation.

This combination makes GCN both efficient and effective for semi-supervised node classification tasks.

2.1.3 Graph Attention Network (GAT)

The Graph Attention Network (GAT) extends the idea of GCN by introducing a learnable attention mechanism that assigns different importance weights to different neighbors.

Instead of treating all neighbors equally, GAT computes attention coefficients:

$$\alpha_{u,v} = \frac{\exp(a^\top [Wh_u \| Wh_v])}{\sum_{v' \in N(u)} \exp(a^\top [Wh_u \| Wh_{v'}])}$$

where:

- W is a shared linear transformation,
- a is a learnable attention vector,
- $\|$ denotes concatenation.

The new representation of node u is then computed as:

$$h_u^{(k)} = \sigma \left(\sum_{v \in N(u)} \alpha_{u,v} Wh_v^{(k-1)} \right)$$

Multi-head attention: GAT uses multiple attention heads to stabilize learning and capture different aspects of the neighborhood. Outputs from different heads are concatenated or averaged.

Key difference with GCN:

- GCN: fixed normalization weights based on graph structure,
- GAT: adaptive weights learned through attention.

Advantages:

- better expressiveness,
- adaptive neighbor importance,
- improved performance on heterogeneous neighborhoods.

2.1.4 Graph Isomorphism Network (GIN)

The Graph Isomorphism Network (GIN) is designed to maximize the expressive power of Graph Neural Networks. Unlike previous architectures such as GCN or GAT, GIN is explicitly motivated by theoretical results showing that many GNN variants fail to distinguish certain graph structures, particularly when using mean or max aggregation functions.

In particular, mean and max aggregators may map different neighborhood structures to the same embedding, limiting their ability to capture structural differences. In contrast, the sum aggregator has stronger discriminative power and can distinguish different multisets of neighbor features.

Based on this observation, GIN adopts a simple but powerful update rule:

$$h_u^{(k)} = \text{MLP}^{(k)} \left((1 + \epsilon^{(k)})h_u^{(k-1)} + \sum_{v \in N(u)} h_v^{(k-1)} \right)$$

where:

- $\epsilon^{(k)}$ is either a learnable parameter or a fixed scalar,
- $\text{MLP}^{(k)}$ is a multi-layer perceptron,
- the sum aggregation ensures injective neighborhood aggregation under mild conditions.

Key idea: GIN increases the expressive power of GNNs by combining sum aggregation with an MLP, allowing it to better distinguish different graph structures.

To obtain a graph-level representation, GIN aggregates representations from all layers using a readout function:

$$h_G = \text{CONCAT}(\{\text{READOUT}(\{h_v^{(k)} | v \in G\}) \mid k = 0, 1, \dots, K\})$$

where the READOUT function typically uses summation over node embeddings.

Intuition: GIN can be interpreted as a GNN that approximates the discriminative power of the Weisfeiler-Lehman graph isomorphism test, making it one of the most expressive architectures among message-passing GNNs [3].

Summary: GIN improves over previous models by:

- using sum aggregation (instead of mean or max),
- employing MLPs instead of linear transformations,
- achieving stronger theoretical expressiveness.

Chapter 3

Experimental Study

3.1 Objective

The main objective of this experimental study is to reproduce and analyze the performance of three representative Graph Neural Network (GNN) architectures, namely Graph Convolutional Networks (GCN) [1], Graph Attention Networks (GAT) [2], and Graph Isomorphism Networks (GIN) [3].

The study is structured into two main stages:

- Reproduction of the results reported in the original papers using standard benchmark datasets.
- Evaluation of the same models on additional datasets provided during the internship in order to assess their generalization capability.

The performance of the models is evaluated using the accuracy metric on node classification and graph classification tasks depending on the dataset.

3.2 Experimental Environment

3.2.1 Hardware and Software Environment

All experiments were conducted using containerized environments (Docker), ensuring full reproducibility across models (GCN, GAT, and GIN). Each model was executed in a Python 3.7-based environment with slightly different dependencies depending on the original implementation.

Component	GCN	GAT	GIN
Base Image	Python 3.7-slim	Python 3.7-slim	Python 3.7-slim
Deep Learning Framework	TensorFlow 1.15.4 + PyTorch 1.13 (CPU)	TensorFlow 1.15.4 + PyTorch 1.13 (CPU)	PyTorch 1.0.0
Graph Library	NetworkX 2.2	NetworkX 2.2	NetworkX (latest)
Graph Deep Learning	PyTorch Geometric	PyTorch Geometric	PyTorch Geometric (limited use)
Scientific Stack	NumPy 1.21, SciPy 1.7	NumPy 1.21, SciPy 1.7	NumPy, SciPy (minimal)
Utilities	Scikit-learn, tqdm	Scikit-learn, pandas, tqdm	Scikit-learn, tqdm
System Dependencies	BLAS, LAPACK, gfortran	BLAS, LAPACK, gfortran	build-essential only

Table 3.1: Software stack used across GCN, GAT, and GIN implementations (Docker-based environments).

3.3 Datasets

3.3.1 Datasets from Original Papers

The following benchmark datasets were used to reproduce the results reported in the original papers.

Dataset	# Nodes	# Edges	# Features	# Classes
Cora	2,708	5,278	1,433	7
Citeseer	3,327	4,552	3,703	6
PubMed	19,717	44,324	500	3

Table 3.2: Node classification datasets used in GCN and GAT reproduction experiments.

These datasets correspond to citation networks where nodes represent scientific publications and edges represent citation links.

For graph classification (GIN experiments), the following datasets were used:

Dataset	# Graphs	# Classes	Avg Nodes
IMDB-BINARY	1000	2	19.8
IMDB-MULTI	1500	3	13.0
REDDIT-B	2000	2	429.6
REDDIT-M5K	5000	5	508.5
COLLAB	5000	3	74.5
MUTAG	188	2	17.9
PROTEINS	1113	2	39.1
PTC	344	2	25.5
NCI1	4110	2	29.8

Table 3.3: Graph classification datasets used for GIN experiments (Xu et al.).

3.3.2 Other Datasets

In addition to classical benchmarks, several new datasets provided by the supervisors were used to evaluate the generalization capability of GCN and GAT models.

Dataset	# Nodes	# Edges	# Classes	Domain
Cornell	183	280	5	Web pages
Texas	183	295	5	Web pages
Wisconsin	251	466	5	Web pages
Squirrel	5,201	198,493	5	Wikipedia network
Chameleon	2,277	31,421	5	Wikipedia network
Actor	7,600	26,752	5	Co-occurrence network
Roman-empire	22,662	32,927	18	Linguistic graph
Amazon-ratings	24,492	93,050	5	Product network
Minesweeper	10,000	39,402	2	Synthetic grid
Tolokers	11,758	519,000	2	Crowdsourcing network
Questions	48,921	153,540	2	User interaction network

Table 3.4: Additional datasets provided during the internship (heterophily and real-world graphs).

3.4 Evaluation Metric

3.4.1 Accuracy

The performance of all models is evaluated using **accuracy**, defined as:

$$\text{Accuracy} = \frac{\text{Number of correct predictions}}{\text{Total number of samples}}$$

Chapter 4

Experimental Results

4.1 Reproduction of GCN Results

Model	Cora	Citeseer	Pubmed	Cornell	Texas	Wisconsin	Chameleon	Squirrel	Actor	Roman-Empire	Amazon
GCN-Paper	0.815	0.703	0.790	-	-	-	-	-	-	-	-
GCN-Our-Implement	0.816	0.713	0.786	0.7669	0.7083	0.6636	0.4894	0.3230	0.3681	0.2559	0.3817
GCN-Our-Implement-H	-	-	-	-	0.7186	-	0.4476	0.3159	-	-	-

Table 4.1: GCN reproduction and evaluation results across standard and heterophilic datasets.

From the results presented in the table, we observe that the reproduction of GCN on citation network datasets (Cora, Citeseer, and PubMed) yields performance that is very close to the results reported in the original paper, confirming the correctness and consistency of our implementation. On the Texas dataset, removing a class containing a single node slightly improves the accuracy, suggesting that isolated or under-represented classes can negatively affect the learning process. In contrast, after harmonizing the Chameleon and Squirrel datasets by removing quasi-duplicates using clustering techniques, we observe a decrease in performance on both datasets. This indicates that these duplicated or highly similar nodes may have artificially inflated the model performance by introducing redundancy that is easier for the GCN to exploit.

4.2 Reproduction of GAT Results

Model	Cora	Citeseer	Pubmed	Cornell	Texas	Wisconsin	Chameleon	Squirrel	Actor	Roman-Empire	Amazon
GAT-Paper	0.830	0.725	0.790	-	-	-	-	-	-	-	-
GAT-Our-Implement	0.8310	0.7189	0.7860	0.4540	0.5536	0.5438	0.5745	0.3528	0.2618	-	-
GAT-Our-Implement-H	-	-	-	-	0.5569	-	0.5552	0.3546	-	-	-

Table 4.2: GAT reproduction results compared with Veličković et al. (2018) and evaluation on datasets.

The results obtained from our GAT reproduction are consistent with the values reported in the original paper for the three standard citation network datasets (Cora, Citeseer, and PubMed), showing only minor variations. This confirms the correctness of our implementation.

After dataset harmonization, we observe different behaviors on heterophilic datasets. In particular, on Chameleon, a slight decrease in performance is observed, while on Squirrel a small improvement is obtained. This suggests that removing noisy or quasi-duplicate structures affects each dataset differently, either by reducing useful structural redundancy or by improving graph quality depending on the case.

4.3 Reproduction of GIN Results

Model	IMDB-B	IMDB-M	REDDIT-B	REDDIT-M5K	COLLAB	MUTAG	PROTEINS	PTC	NCI1
GIN-Paper (GIN-0)	0.751	0.523	0.924	0.575	0.802	0.894	0.762	0.646	0.827
GIN-Paper (GIN- ϵ)	0.743	0.521	0.922	0.570	0.801	0.890	0.759	0.637	0.827
GIN-Our-Implement	0.770	0.527	0.795	0.524	-	0.947	0.795	0.800	0.827

Table 4.3: GIN reproduction results compared with Xu et al. (2019).

The results obtained from our GIN implementation are generally consistent with those reported in the original paper (Xu et al., 2019), particularly on the IMDB-B and IMDB-M datasets, where we observe very close performance to both GIN-0 and GIN- ϵ variants. This indicates that our reproduction correctly captures the main behavior of the model.

However, some noticeable differences can be observed on certain datasets such as REDDIT-B and REDDIT-M5K, where our implementation shows lower performance compared to the reported results. These gaps may be attributed to implementation details, training setup, or hyperparameter sensitivity, which are known to significantly affect GIN performance on large-scale graph classification tasks.

Overall, these results confirm that our implementation is reliable while also highlighting the sensitivity of GIN to dataset characteristics and experimental configurations.

Bibliography

- [1] T. N. Kipf and M. Welling, “Semi-Supervised Classification with Graph Convolutional Networks,” Feb. 2017, arXiv:1609.02907 [cs.LG]. [Online]. Available: <http://arxiv.org/abs/1609.02907>
- [2] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Lio, and Y. Bengio, “Graph attention networks,” in *International conference on learning representations*, vol. 6, no. 2. Ithaca, 2018. [Online]. Available: <https://www.chapterpal.com/s/vq6tnqa4/graph-attention-networks>
- [3] K. Xu, W. Hu, J. Leskovec, and S. Jegelka, “How Powerful are Graph Neural Networks?” Feb. 2019, arXiv:1810.00826 [cs.LG]. [Online]. Available: <http://arxiv.org/abs/1810.00826>